Appendix. Bootstrapping Spring applications

You can kick-start your Spring projects in a lot of ways, and which you choose is largely a matter of personal taste. Many of the choices will be decided by which IDE is your favorite.

All but one of these options are based on the Spring Initializr, which is a REST API that generates Spring Boot projects for you. The various IDE choices are nothing more than clients for that REST API. Additionally, you have a few ways to use the Spring Initializr API outside of your IDE. This appendix takes a quick look at all of these options.

A.1 Initializing a project with Spring Tool Suite

To initialize a new Spring project with Spring Tool Suite, choose the Spring Starter Project menu option from the File > New menu, as shown in figure A.1.

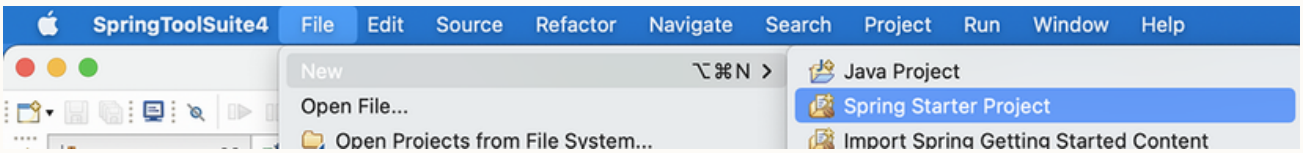


Figure A.1 Starting a new project in Spring Tool Suite

NOTE This is an abbreviated description of using Spring Tool Suite to initialize a Spring project. For a more detailed explanation, see section 1.2.1.

You'll be shown the first page of the project creation dialog box (figure A.2). On this page, you'll define basic project information, such as the project's name, coordinates (group ID and artifact ID), version, and base package name. You can also specify whether the project will be built with Maven or Gradle, whether the build will produce a JAR file or a WAR file, which version of Java to build with, and even an alternate JVM language to use, such as Groovy or Kotlin.

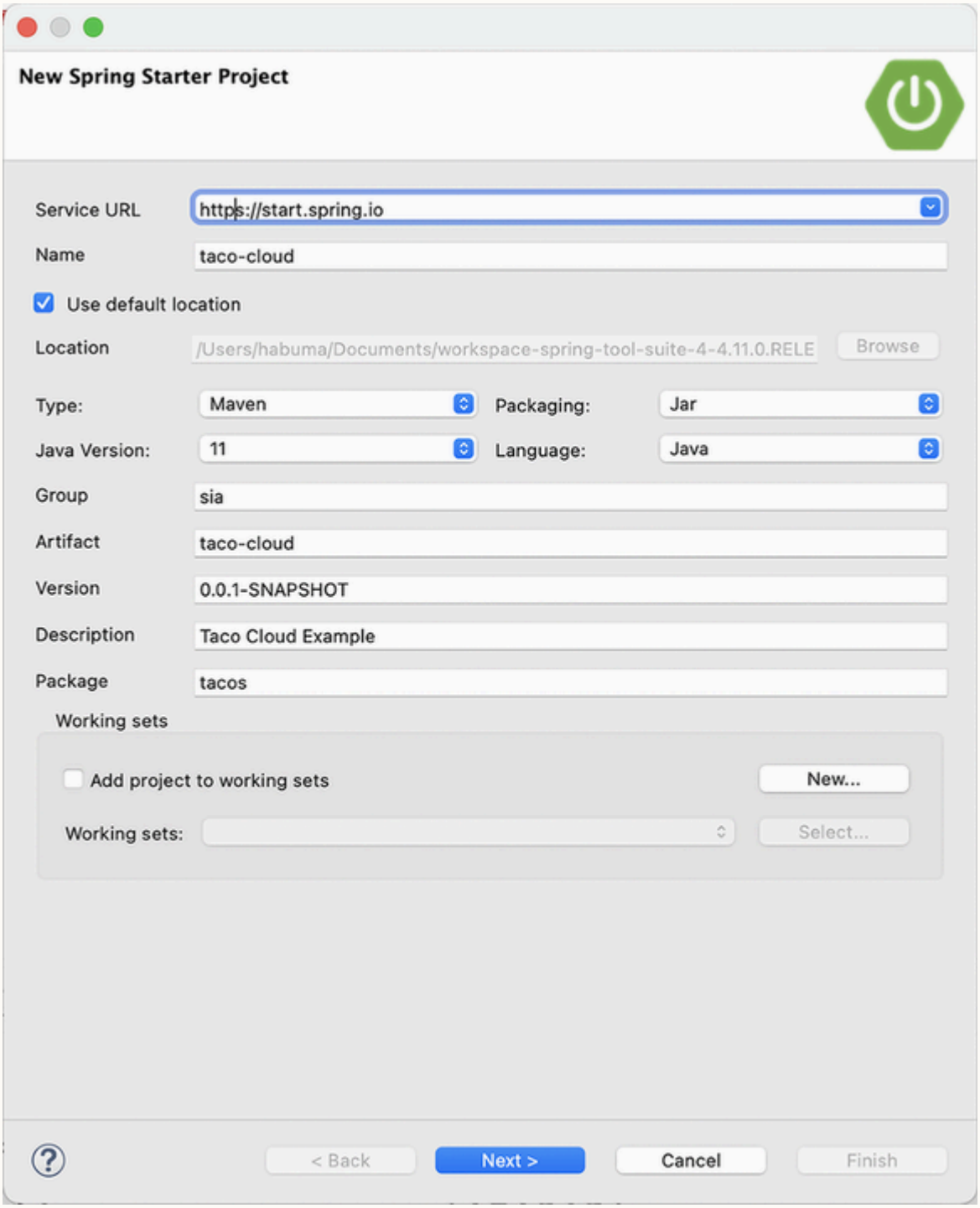


Figure A.2 Defining basic project information

The first field on this page asks you to specify the location of the Spring Initializr service. If you're running or using a custom instance of the Initializr, you'll want to specify the base URL of the Initializr service here. Otherwise, you'll be fine leaving it with the default that points to **http://start.spring.io**.

After you've defined the basic project information, click Next to see the project dependencies page (see figure A.3).

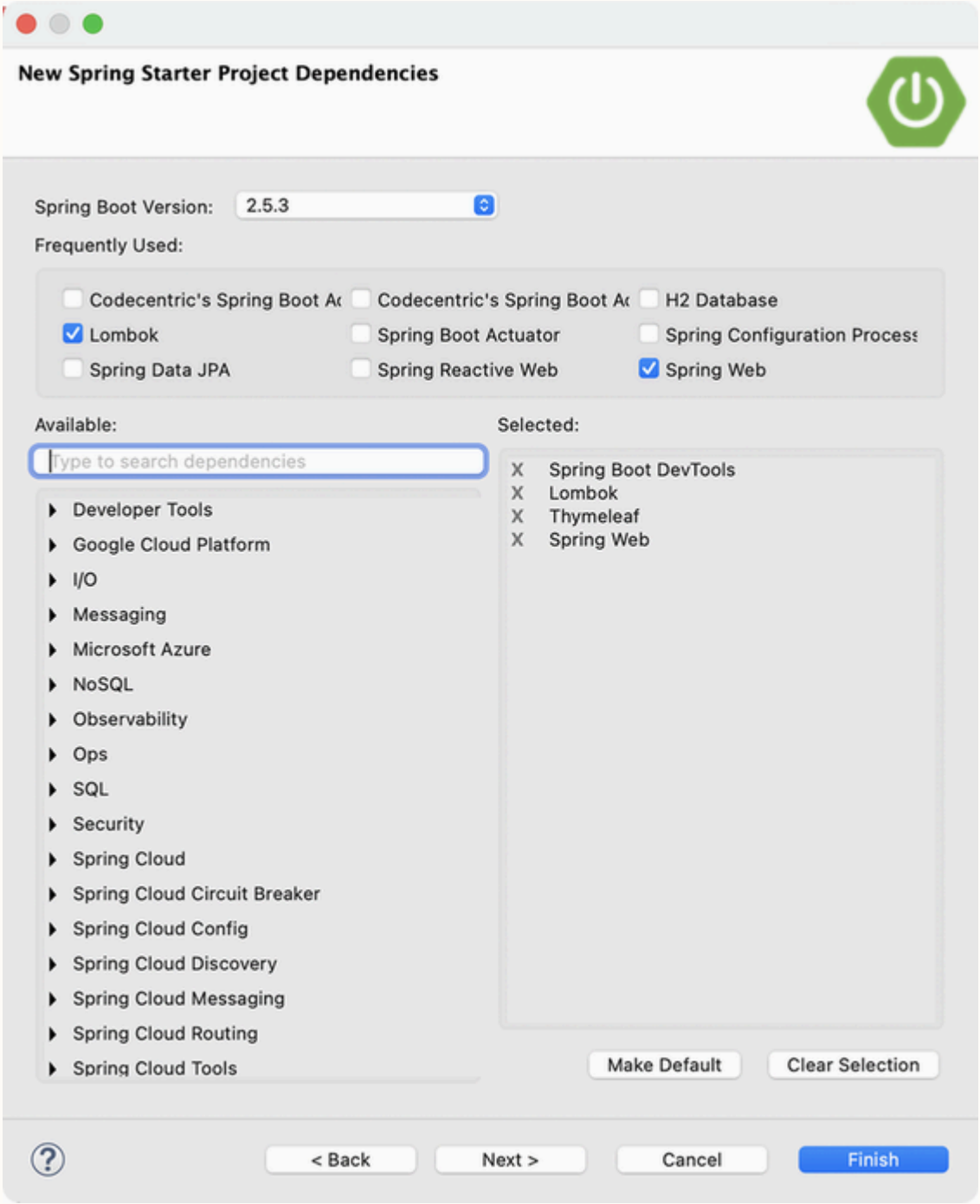


Figure A.3 Specifying project dependencies

On the project dependencies page, you can specify all of the dependencies your project will need. Many of these dependencies are Spring Boot Starter dependencies, although some other dependencies are commonly used in Spring projects.

The available dependencies are listed on the left side, organized in groups that can be expanded or collapsed. If you're having trouble finding a dependency, you can also search for dependencies to narrow down your choices.

To add a dependency to the generated project, select the check box next to the dependency name. Your selections will appear in the list on the right side under the Selected header. You can remove a dependency by clicking the X next to the selected dependency, or click Clear Selection to remove all selected dependencies.

As an added convenience, if you find that you have a certain core set of dependencies that you always (or often) use for your projects, you can click the Make Default button after selecting those dependencies, and they'll already be selected the next time you create a project.

After making your selections, click Finish to generate the project and add it to your workspace. If, however, you want to use an Initializr other than the one at http://start.spring.io, click Next to set the Initializr base URL, as shown in figure A.4.

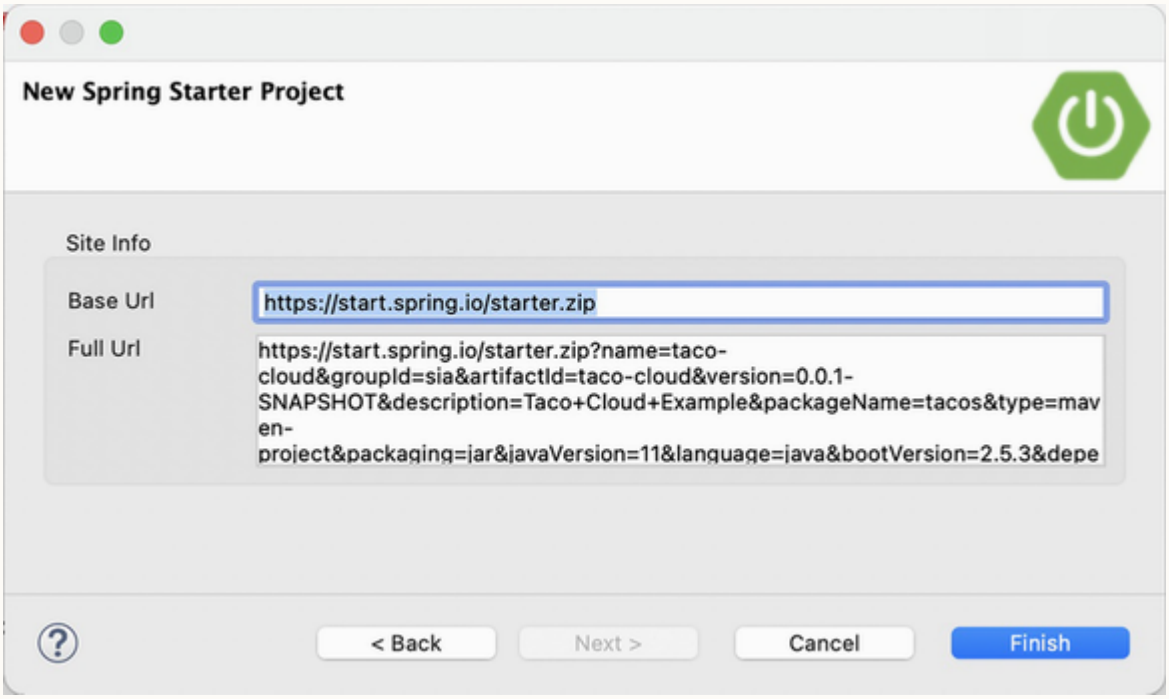


Figure A.4 Optionally specifying the Initializr base URL

The Base Url field specifies the URL where the Initializr API is listening. This is the only field you can change on this page. The Full Url field shows the complete URL that will be used to request a new project from the Initializr.

A.2 Initializing a project with IntelliJ IDEA

To get started on a new Spring project in IntelliJ IDEA, choose the Project menu item from the File > New menu, as shown in figure A.5.

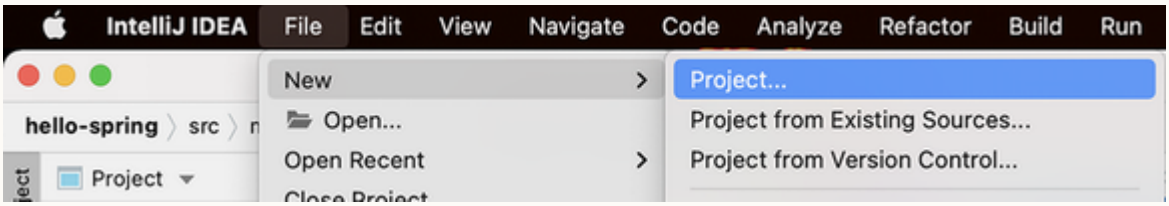


Figure A.5 Starting a new Spring project in IntelliJ IDEA

This opens up the first page of a new Spring Initializr project wizard. You'll be presented with a page that asks for essential project information, as shown in figure A.6. You may recognize some of the fields on this page as information that might appear in a Maven pom.xml file—in fact, if you select Maven Project from the Type field, that's exactly how it will be used. You're welcome to choose Gradle Project instead if Gradle is your preference.

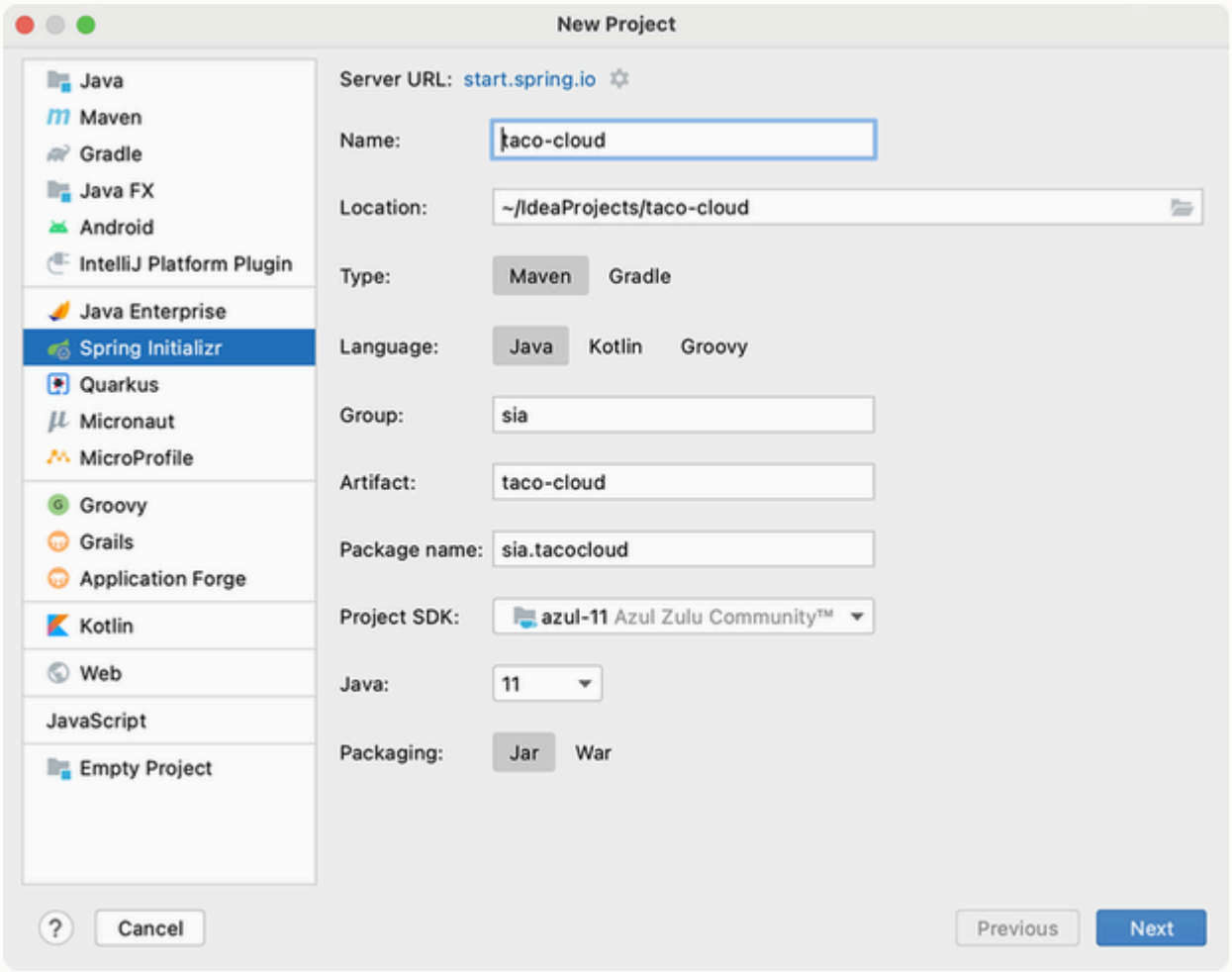


Figure A.6 Specifying essential project information in IntelliJ IDEA

Once you've filled in the essential project information, click Next to be shown the project dependencies page (see figure A.7).

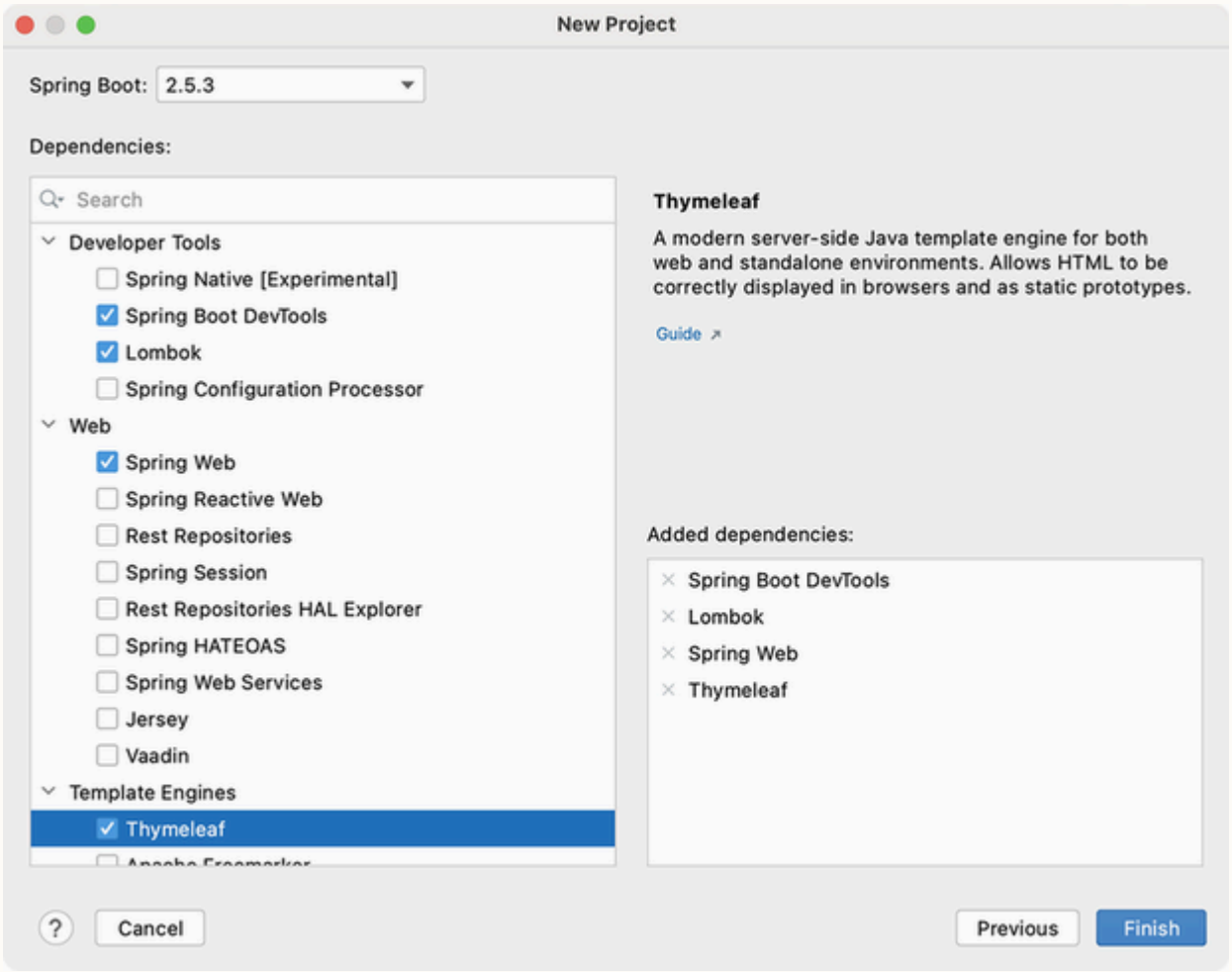


Figure A.7 Selecting project dependencies

The dependencies are organized by category in the far-left list. Selecting a category will result in that category's options being presented in the middle list. Your selected dependencies will be listed (according to category) in the right list.

After all of your dependencies have been selected, click Finish. Your project will be created and loaded into the IntelliJ IDEA workspace.

A.3 Initializing a project with NetBeans

Before you can create a new Spring Boot project in NetBeans, you need to install a plugin that enables Spring Boot development in NetBeans. The NB Spring Boot plugin adds features to NetBeans that are similar to those built into Spring ToolSuite and IntelliJ IDEA.

To install the plugin, select the Plugins option from the Tools menu, as shown in figure A.8.

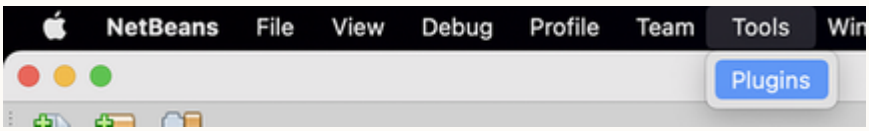


Figure A.8 The NetBeans Plugins menu item

You'll be shown a list of available plugins for NetBeans, including the NB Spring Boot plugin, as shown in figure A.9.

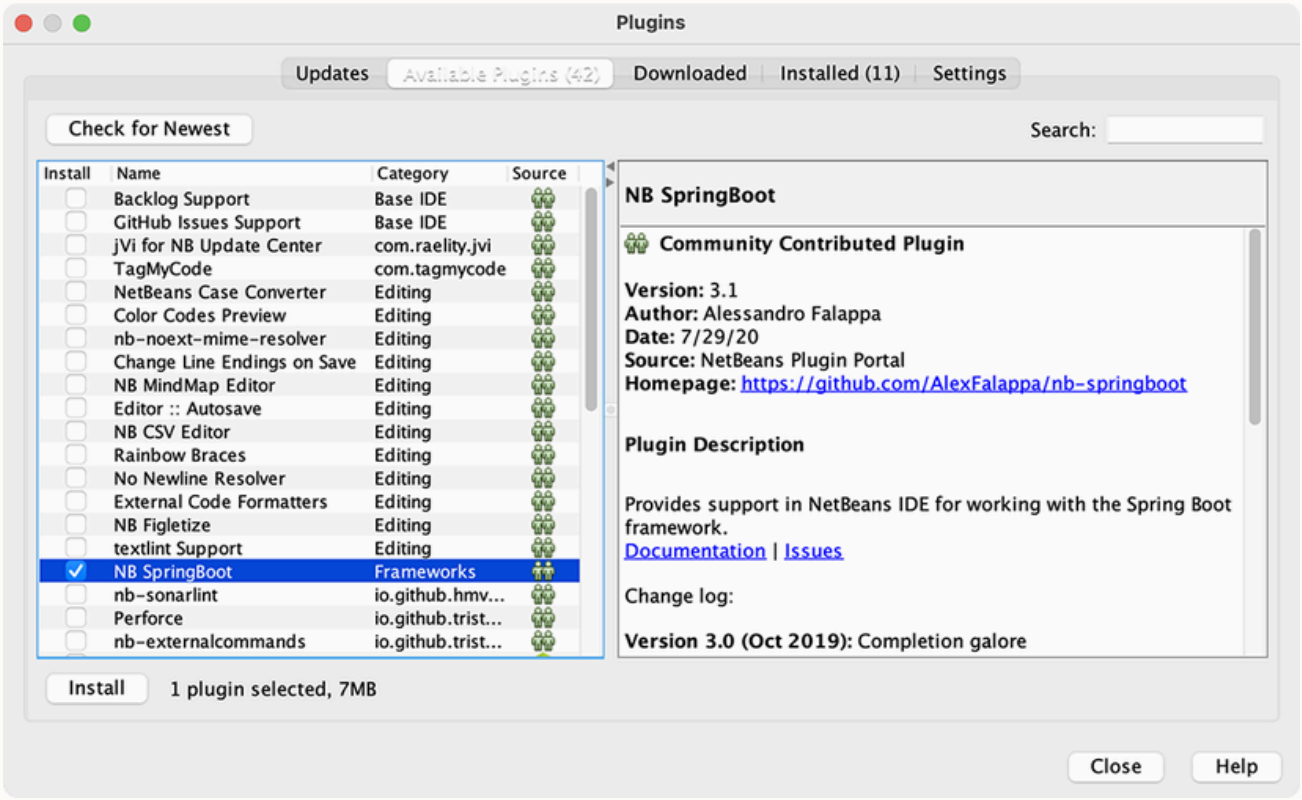


Figure A.9 Selecting the NB Spring Boot plugin

Click Install to begin the installation of the Spring Boot plugin. You'll be prompted with a handful of dialogs to confirm your decision and to acknowledge the plugin license agreement. Simply click Next through them all until you get to the last one, then click Install. Finally, you'll be prompted to restart NetBeans for the plugin to take effect.

After installing the Spring Boot plugin, you are ready to initialize a new Spring Boot project in NetBeans. To create a new Spring project in NetBeans, start by selecting the New Project menu item under the File menu, as shown in figure A.10.

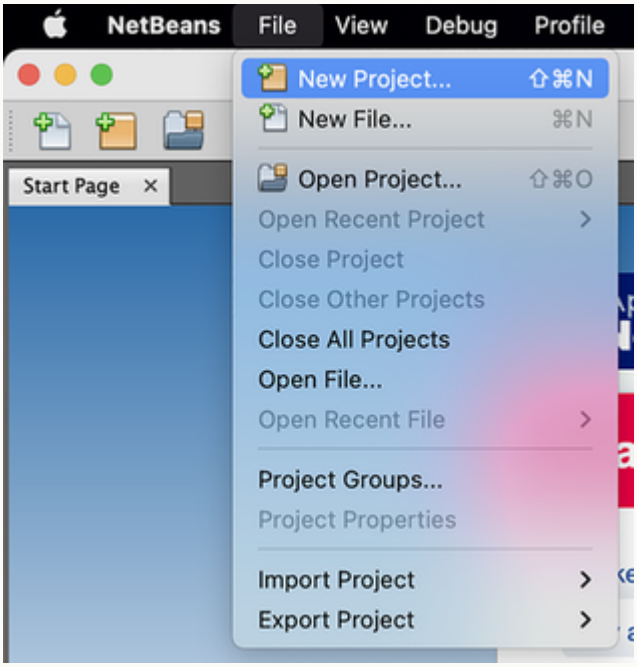


Figure A.10 Starting a new Spring project in NetBeans

You'll be shown the first page of the new project wizard. As shown in figure A.11, this page will let you choose what kind of project you want to create.

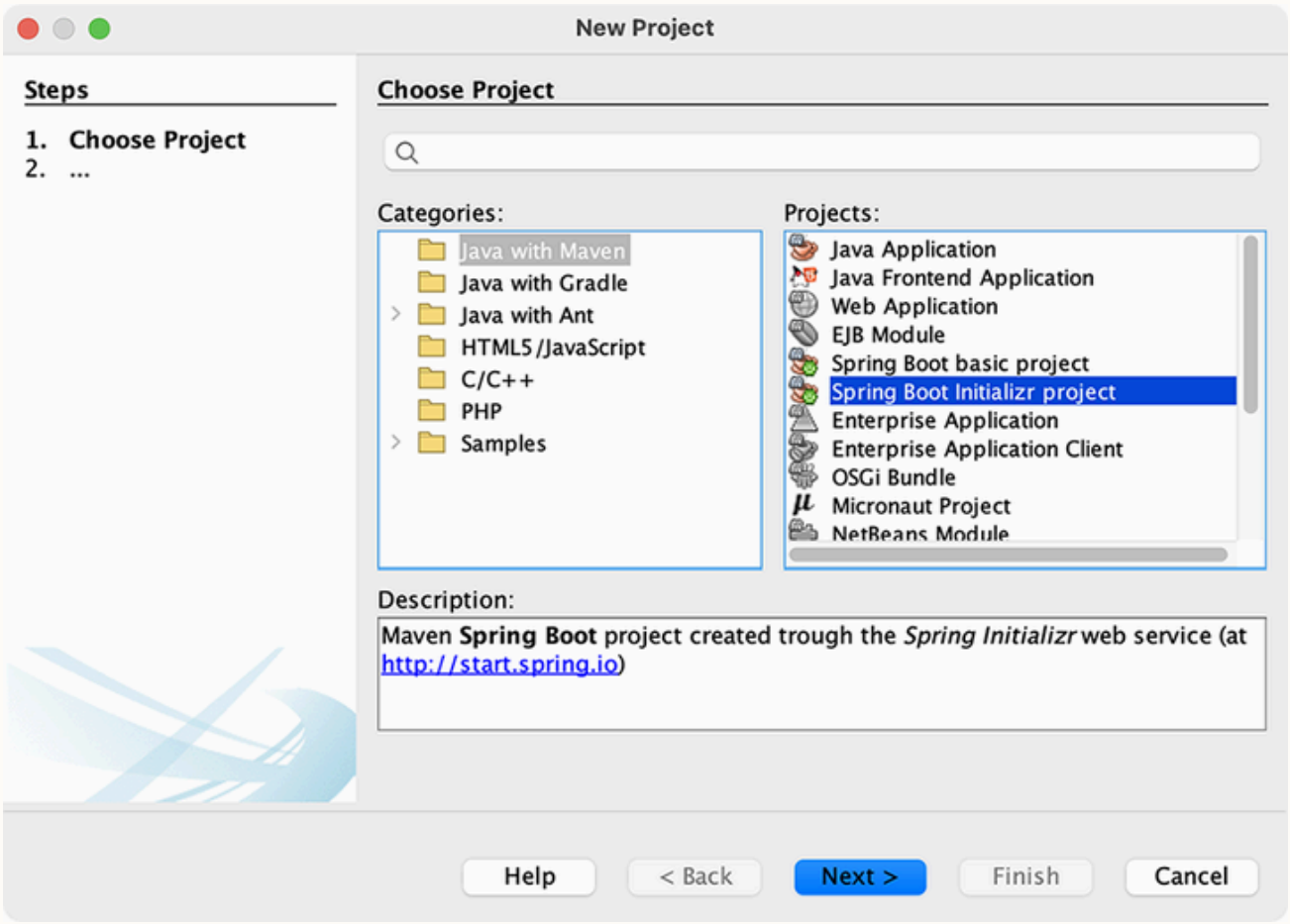


Figure A.11 Creating a new Spring Boot Initializr project

For a Spring Boot project, select Java with Maven from the category list on the left, and then select Spring Boot Initializr Project from the project list on the right. Then click

Next to move to the next page.

The second page in the new project wizard (figure A.12) lets you set essential project information, such as the project name, version, and other information that will ultimately be used to define the project in a Maven pom.xml file.

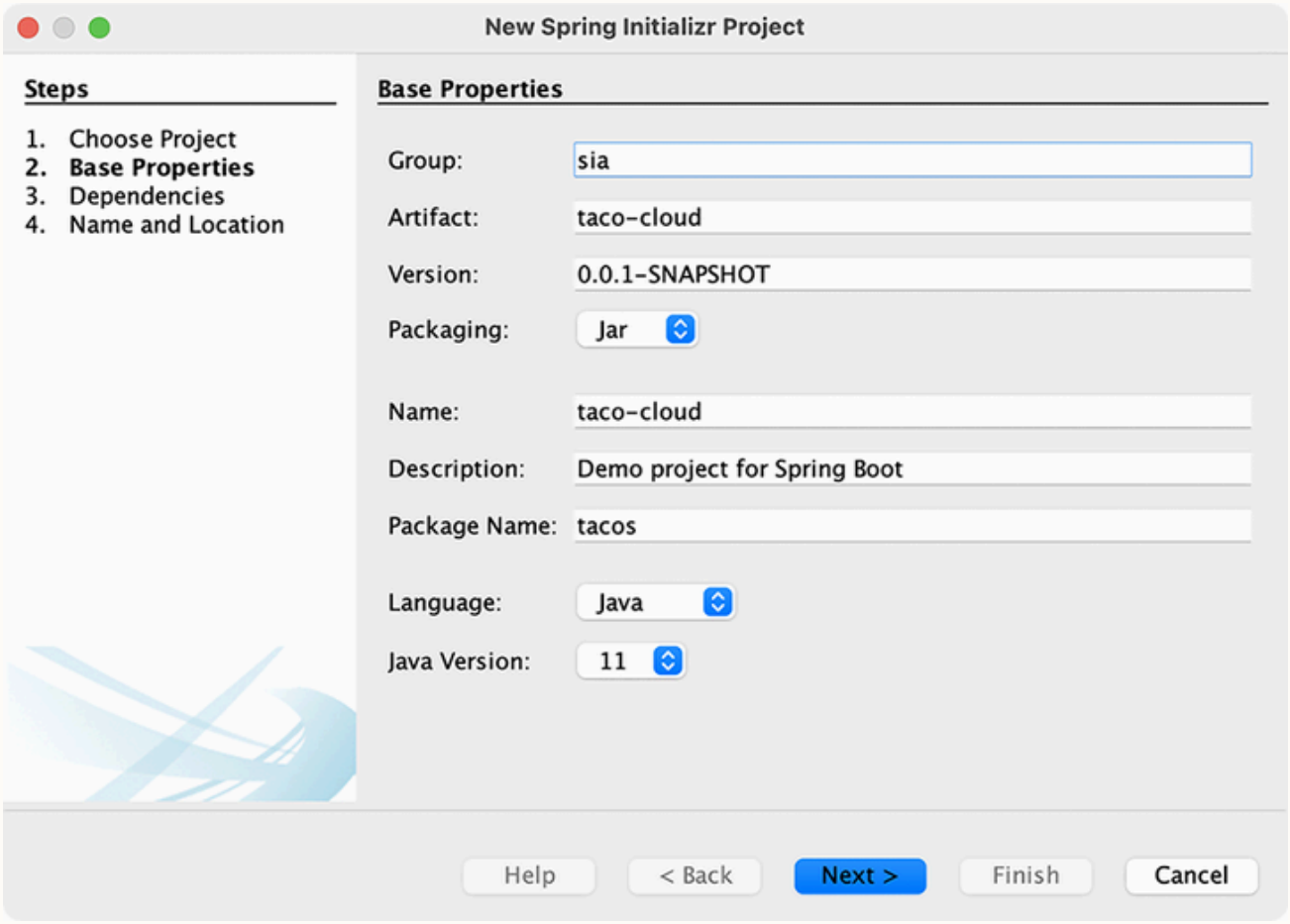


Figure A.12 Specifying essential project information

After you've specified the basic project information, click Next to navigate to the dependencies page in the new project wizard, shown in figure A.13.

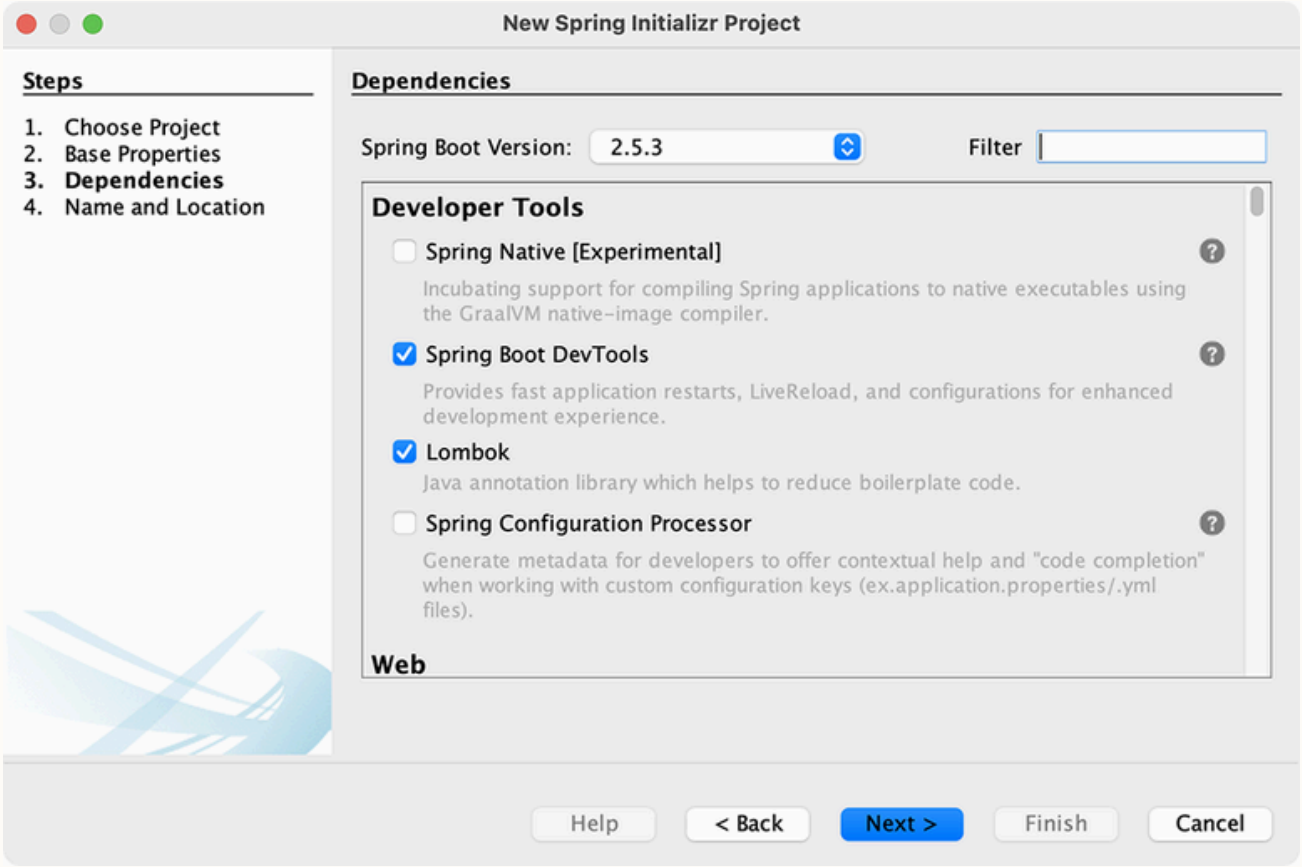


Figure A.13 Selecting project dependencies

Dependencies are all listed as check boxes in the same list, organized by category. If you have trouble finding the specific dependency you need, you can use the Filter text box at the top to limit the list of options.

You can also specify which version of Spring Boot you wish to use on this page. It will be set to the current generally available version of Spring Boot by default.

Once you've selected the dependencies for your project, click Next to navigate to the last page of the new project wizard, shown in figure A.14. This page lets you specify some final details about the project, including the project name and location on the filesystem. (The Project Folder field is read only and derived from the other two fields.) It also gives you the option to run and debug your project through the Maven Spring Boot plugin instead of through NetBeans. You may also choose to have NetBeans remove the Maven wrapper from the generated project.

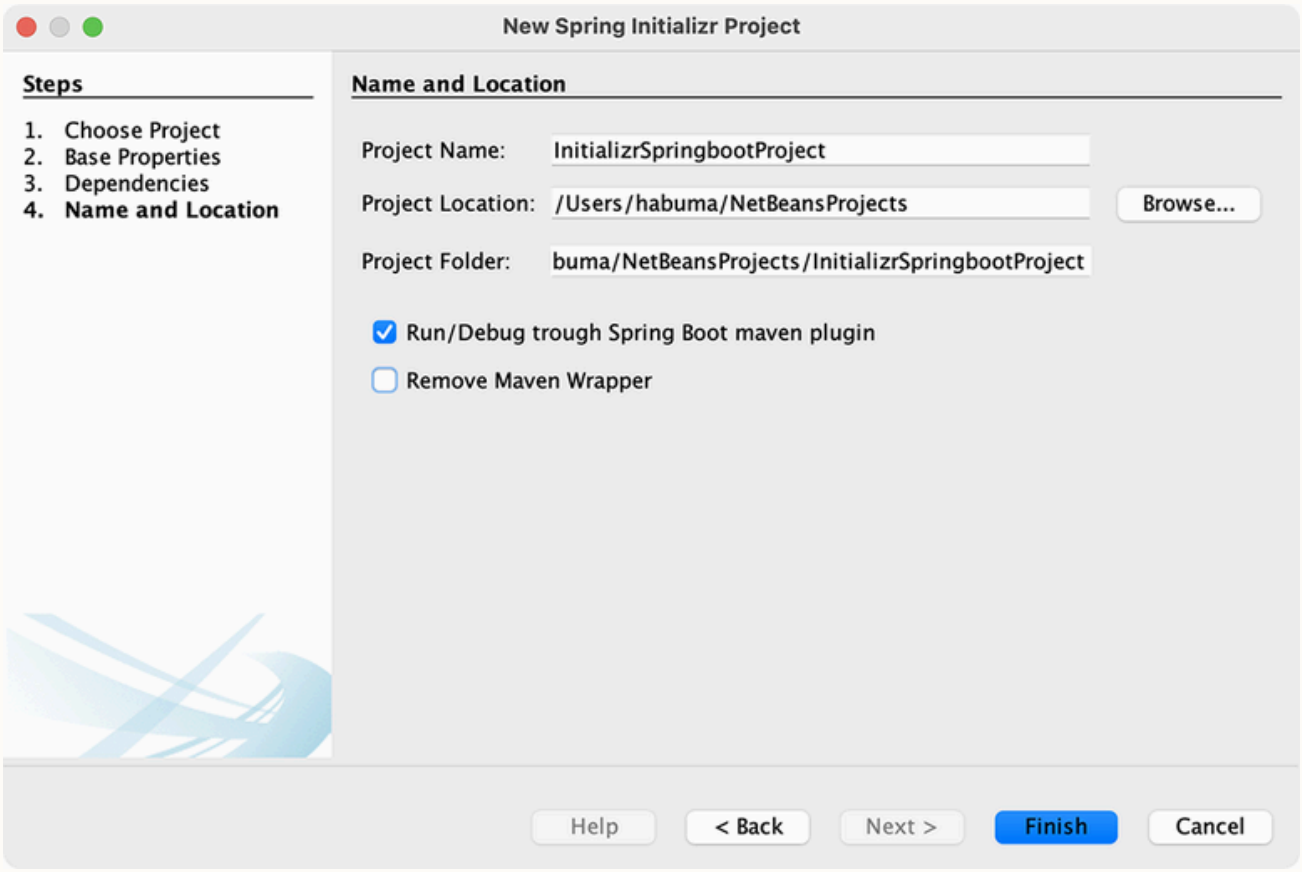


Figure A.14 Specifying the project's name and location

Once you've set the final bit of project information, click Finish to generate the project and have it added to your NetBeans workspace.

A.4 Initializing a project at start.spring.io

Although one of the IDE-based initialization options described thus far will likely suit your needs, it's possible that you may use a completely different IDE, or you might favor working with a simpler text editor. In that case, you can still take advantage of the Spring Initializr using the Initializr web-based interface.

To get started, direct your favorite web browser to **https://start.spring.io**. You should see the simple version of the Spring Initializr web user interface, shown in figure A.15.

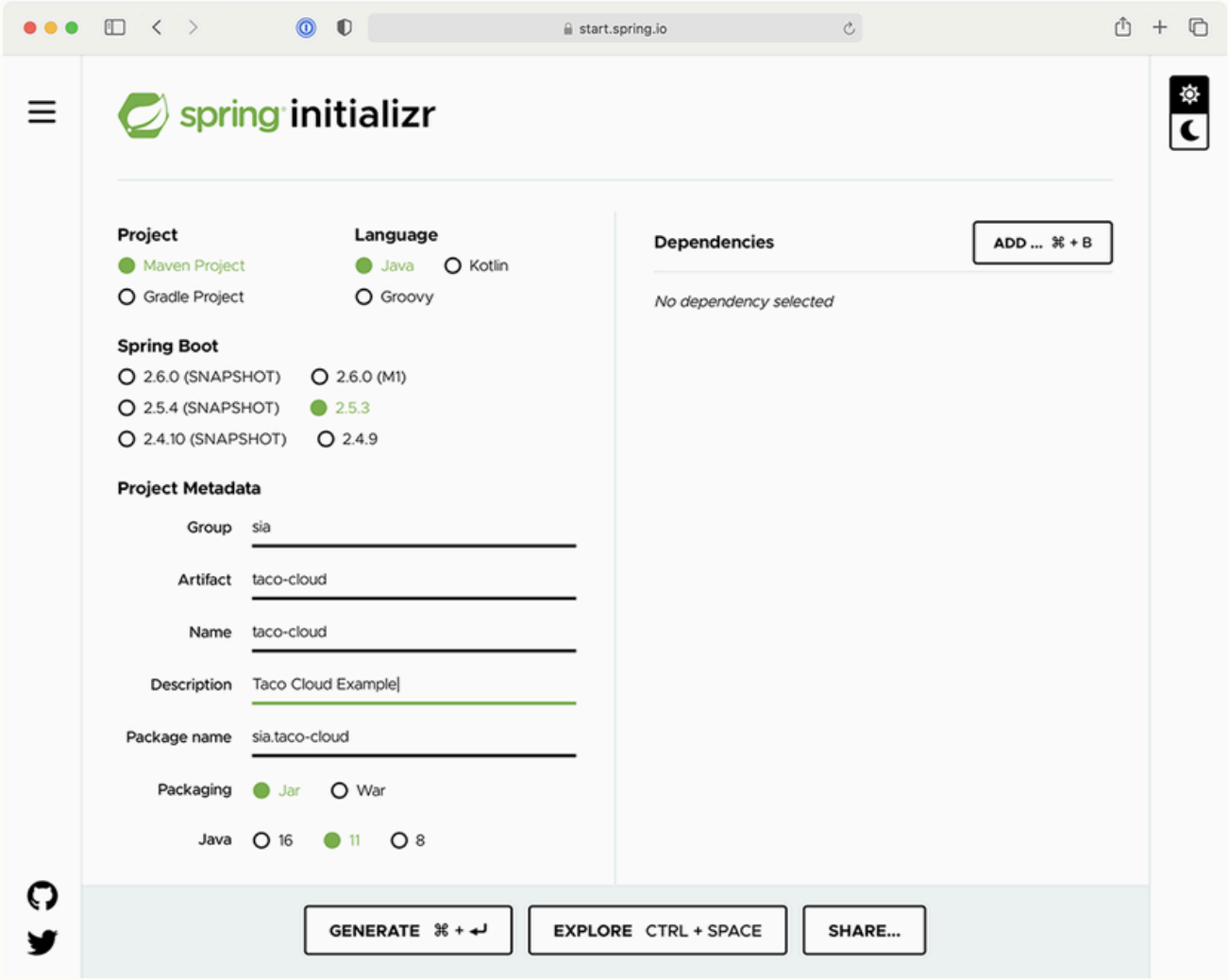


Figure A.15 The simple version of the Spring Initializr web interface

In the simple version of the Initializr web application, you're asked for some very basic information, including whether you want to build with Maven or Gradle, which language you want to develop the project with, which version of Spring Boot to build against, and the group and artifact IDs of the project.

You'll also have the option of specifying dependencies by typing search criteria in the search box. For example, as shown in figure A.16, you can type `web` to search for any dependencies where "web" is a keyword.

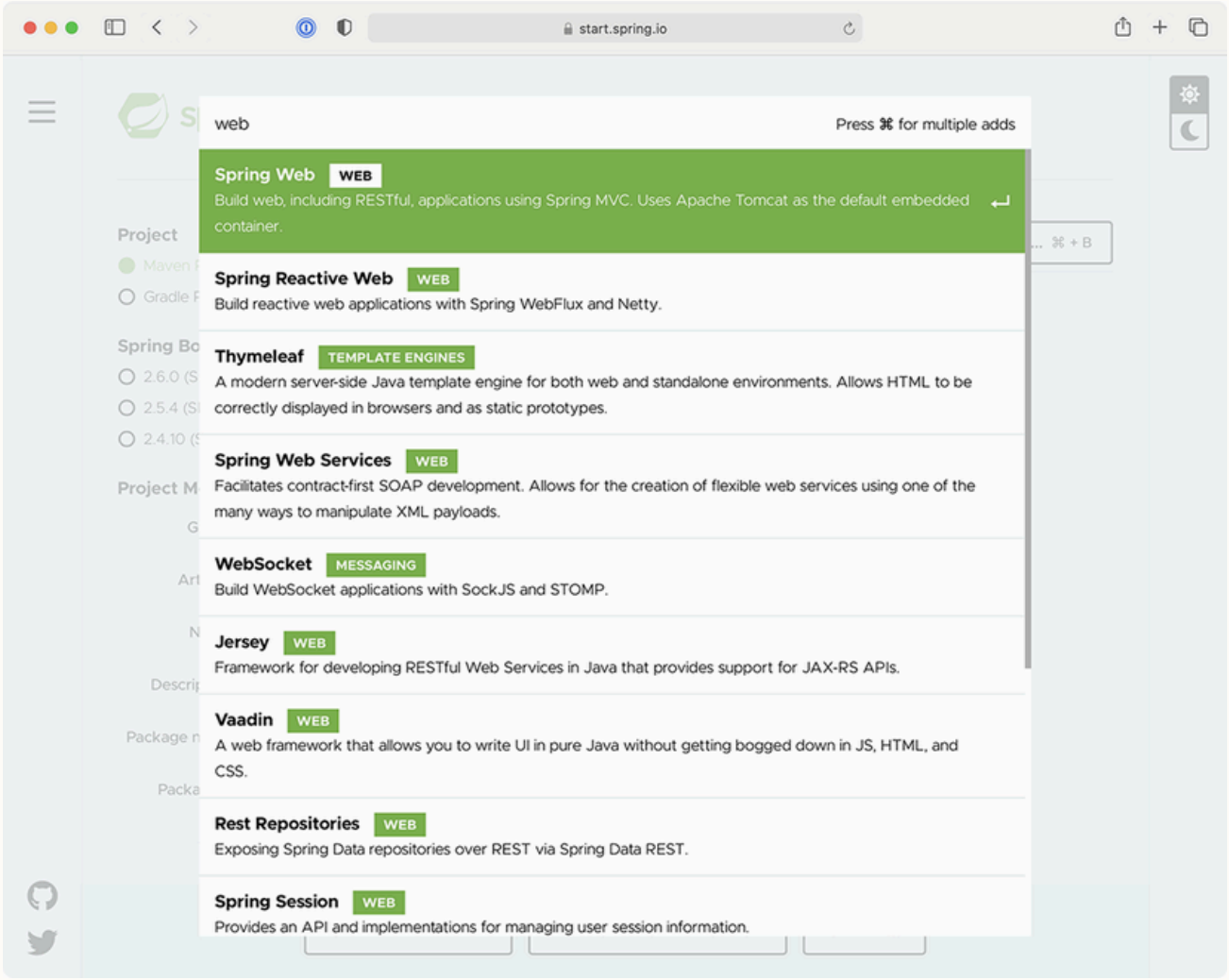


Figure A.16 Searching for dependencies

When you see the dependency you want, press Return on your keyboard to select it, and it will be added to the list of selected dependencies. The boxes beneath Selected Dependencies in figure A.17 show that the Web, Thymeleaf, DevTools, and Lombok dependencies have been selected.

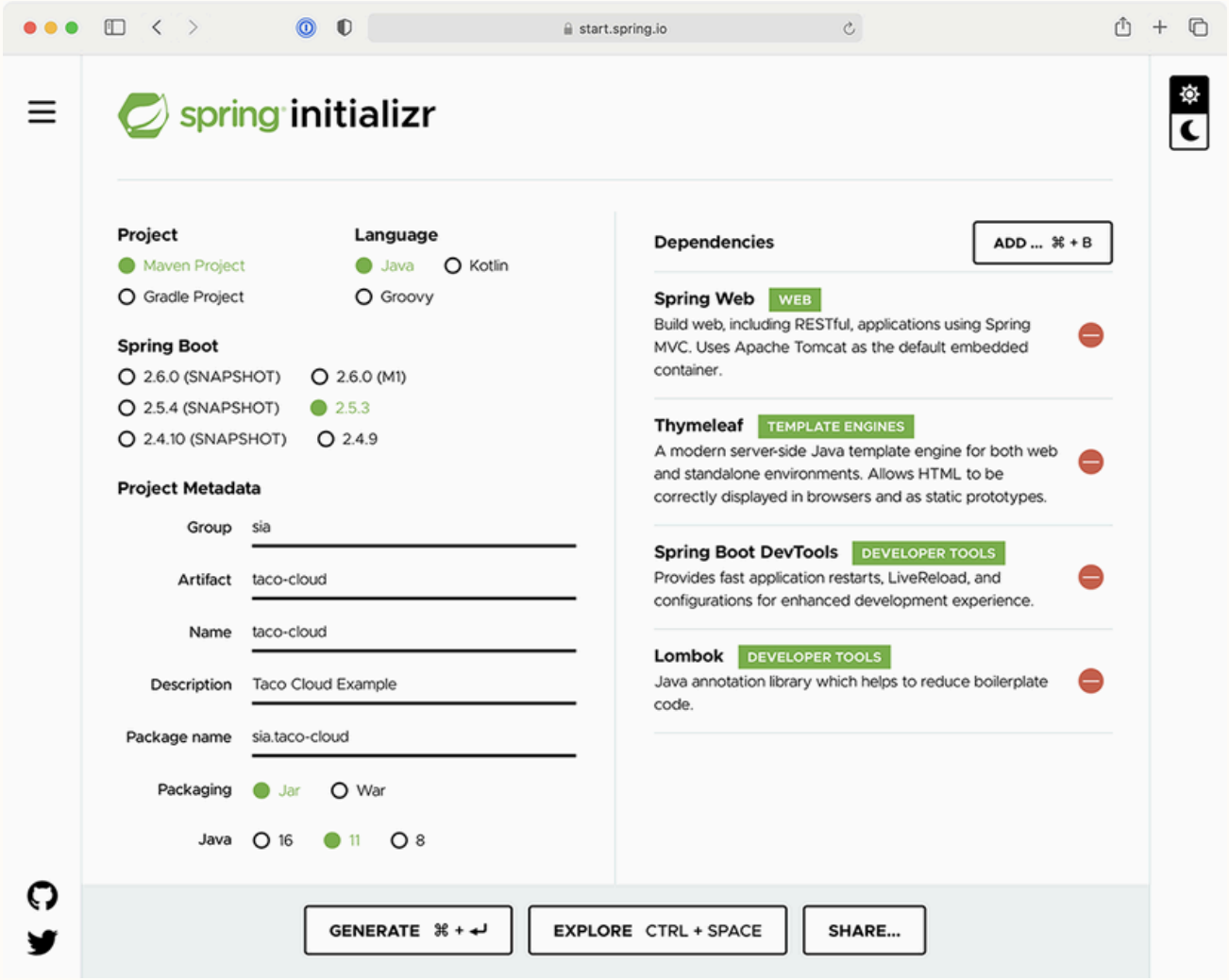


Figure A.17 Selecting dependencies

If you decide you don't need a selected dependency, you can click the X to the right of the dependency entry to remove it. When you're finished, you can click Generate Project (or use the keyboard shortcut displayed on the button, which will vary by operating system) to have the Initializr generate the project and download it as a zip file. Then you can unzip the project and load it in whatever IDE or editor you choose.

Before clicking Generate Project, you can get a sneak peak of the project by clicking Explore. This will pull up a dialog with a project explorer, much like the one shown in figure A.18.

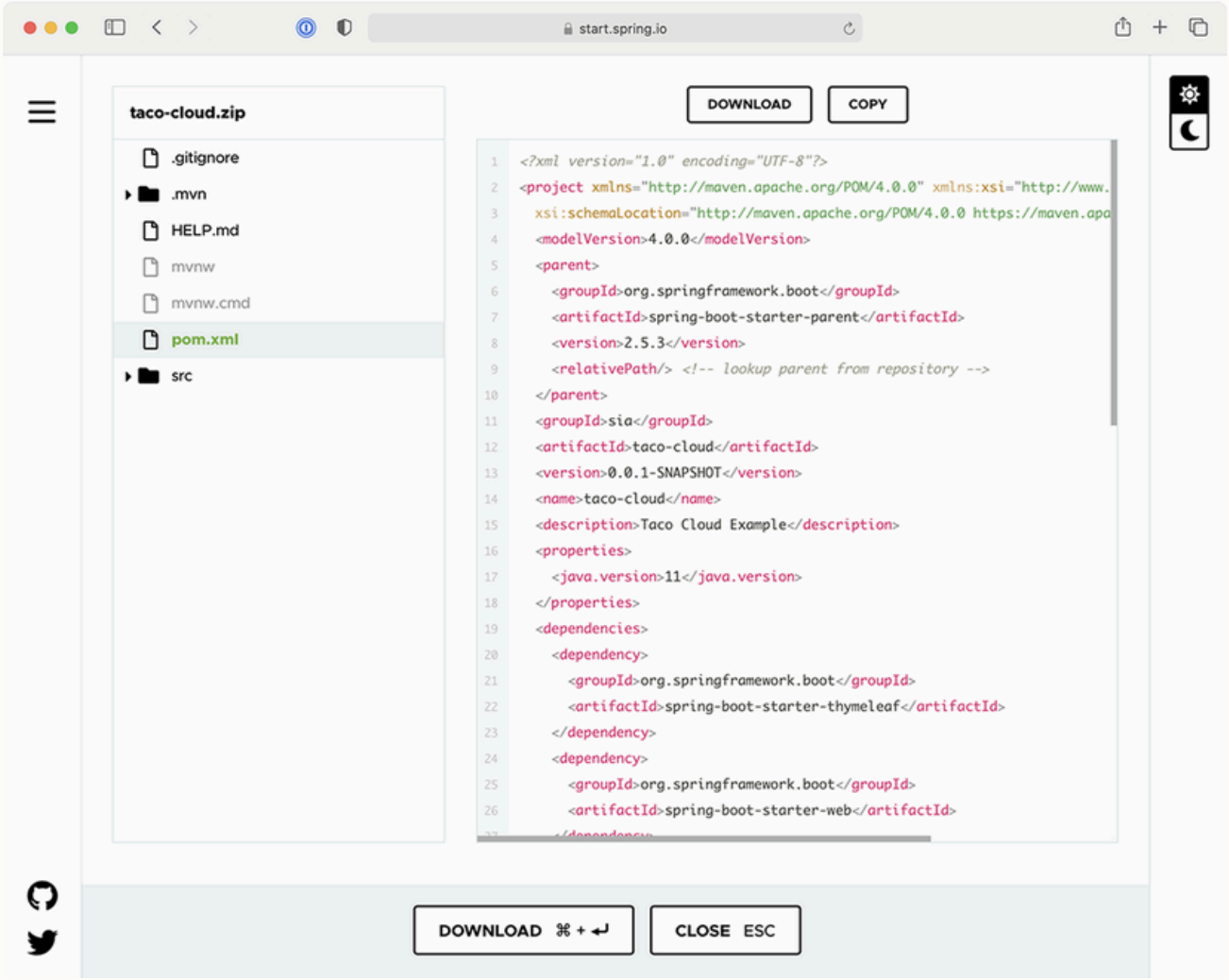


Figure A.18 The full version of the Initializr user interface

The project's build specification (either a Maven pom.xml file or Gradle build.gradle file) will be shown first. By clicking on items in the tree on the left, you can see what other artifacts will be included in the project.

A.5 Initializing a project from the command line

The IDE and browser-based user interfaces for the Spring Initializr are probably the most common way that you'll bootstrap your projects. They're all just clients of a REST service offered by the Initializr application. In some special cases (e.g., in a scripted scenario), you might find it useful to consume the Initializr service directly from the command line.

You can consume the API in the following two ways:

- Using the `curl` command (or some similar command-line REST client command)
- Using the Spring Boot command-line interface (aka, Spring Boot CLI)

Let's look at these options, starting with the `curl` command.

url and the Initializr API

The simplest way to bootstrap a Spring project with `curl` is to consume the API like this:

```
% curl https://start.spring.io/starter.zip -o demo.zip
```

In this case, you're requesting the /starter.zip endpoint from the Initializr, which will generate a Spring project and download it as a zip file. The generated project will be Maven built and will have no dependencies other than the base Spring Boot starter dependency. All project information in the project's pom.xml file will be set to default values.

If you don't specify otherwise, the name of the file will be starter.zip. But in this case, the `-o` option specifies that the downloaded file should be named demo.zip.

The publicly available Spring Initializr server is hosted at **https://start.spring.io**, but if you're using a custom Initializr, you'll need to adapt the given URL accordingly.

You'll probably want to specify a few more details and dependencies beyond the given defaults. Table A.1 lists all of the parameters (and their defaults) when consuming the Spring Initializr REST service.

Table A.1 Request parameters supported by the Initializr API

Parameter	Description	Default value
groupId	The project's group ID, for the sake of organization in a Maven repository	com.example
artifactId	The project's artifact ID, as it would appear in a Maven repository	demo
version	The project version	0.0.1-SNAPSHOT
name	The project name; also used to determine the name of the application's main class (with an `Application` suffix)	demo
description	The project description	Demo project for Spring Boot
packageName	The project's base package name	com.example.demo
dependencies	Dependencies to include in the project's build specification	The base Spring Boot starter
type	The kind of project to generate: either `maven-project` or `gradle-project`	maven-project
javaVersion	The version of Java to build with	1.8
bootVersion	The version of Spring Boot to build against	The current GA version of Spring Boot
language	The programming language to use: `java`, `groovy`, or `kotlin`	java
packaging	How the project should be packaged: either `jar` or `war`	jar

applicationName	The name of the application	The value of the name parameter
baseDir	The name of the base directory in the generated archive	The root directory

You can also get this list of parameters, as well as a list of available dependencies, by making a simple request to the base Initializr URL as follows:

```
% curl https://start.spring.io
```

The `dependencies` parameter is the one you'll probably find the most useful. For example, suppose that you want to create a simple web project with Spring. The following command-line use of `curl` will produce a project zip with the web starter as a dependency:

```
% curl https://start.spring.io/starter.zip \
    -d dependencies=web \
    -o demo.zip
```

As a more complex example, suppose you wanted to develop a web application that uses Spring Data JPA for data persistence. You also want to build it with Gradle, and the project should be under a directory named my-dir within the zip file. And let's suppose that rather than just download a zip file, you want the project unpacked into your filesystem upon download. In that case, the following command should do the trick:

```
% curl https://start.spring.io/starter.tgz \
     -d dependencies=web,data-jpa \
     -d type=gradle-project
     -d baseDir=my-dir | tar -xzvf -
```

Here, the downloaded zip file is piped to the `tar` command for unpacking.

pring Boot command-line interface

The Spring Boot CLI is another option for initializing Spring applications. You can install the Spring Boot CLI in many ways, but probably the easiest way (and my favorite) is to use SDKMAN (**http://sdkman.io/**), as shown next:

```
% sdk install springboot
```

Once the Spring Boot CLI is installed, you can start using it to generate projects, much like with `curl`. The command you'll use is `spring init`. In fact, the simplest way to use the Spring Boot CLI to generate a project is like this:

```
% spring init
```

This will result in a bare-bones Spring Boot project being downloaded in a zip file named demo.zip. However, you'll probably want to specify more details and dependencies. Table A.2 lists all of the parameters available to the `spring init` command.

Table A.2 Request parameters supported by the `spring init` command

Parameter	Description	Default value
group-id	The project's group ID, for the sake of organization in a Maven repository	com.example
artifact-id	The project's artifact ID, as it would appear in a Maven repository	demo
version	The project version	0.0.1-SNAPSHOT
name	The project name; also used to determine the name of the application's main class (with an `Application` suffix)	demo
description	The project description	Demo project for Spring Boot
package-name	The project's base package name	com.example.demo
dependencies	Dependencies to include in the project's build specification	The base Spring Boot starter
type	The kind of project to generate: either `maven-project` or `gradle-project`	maven-project
java-version	The version of Java to build with	11
boot-version	The version of Spring Boot to build against	The current GA version of Spring Boot
language	The programming language to use: `java`, `groovy`, or `kotlin`	java
packaging	How the project should be packaged: either `jar` or `war`	jar

You can also get this list of parameters, as well as a list of available dependencies, by using the `--list` parameter as follows:

```
% spring init --list
```

Suppose you wish to create a web application that builds against Java 1.7. The following command uses the `--dependencies` and `--java` parameters to make those choices:

```
% spring init --dependencies=web --java-version=1.7
```

Or suppose you want to create a web application with Spring Data JPA for persistence, and you'd like to use Gradle to perform the build instead of Maven. You'd use the following command:

```
% spring init --dependencies=web,jpa --type=gradle-project
```

You may also notice that many of the `spring init` parameters are the same as or similar to the parameters for the `curl` option. That said, the `spring init` command doesn't support all of the same parameters as the `curl` option (e.g., `baseDir`), and the parameters are hyphen-delimited instead of camelCase (e.g., `package-name` versus `packageName`).

A.6 Building and running projects

No matter how you initialize your project, you can always run the application from the command line with the `java -jar` command as follows:

```
% java -jar demo.jar
```

This will even work if you decide to create a WAR file distribution instead of a JAR file, as shown here:

```
% java -jar demo.war
```

You can also take advantage of the Spring Boot Maven and Gradle plugins to run your application. For example, if your project is built with Maven, you can run it like this:

```
% mvn spring-boot:run
```

If, on the other hand, you've chosen to build your project with Gradle, you can run your project like this:

```
% gradle bootRun
```

In either case, whether using Maven or Gradle, the build tool will first build your project (if it hasn't already been built) and run it.

Previous chapter
< 18 Deploying Spring

Next chapter
index >